Universität Rostock



SCHLEIFEN

Joern Ploennigs · AI4SC

Titelbild mit konzentrischen Schleifen, generiert mit Midjourney nach einer visuellen Referenz auf Robert Delaunay

Warum Schleifen?

Manuelle Berechnung scheitert an der Datenmenge. Schleifen sind das Werkzeug für diese Skala.

BODENPROBEN

Baustelle

10.000 Proben auswerten

SENSORMESSUNGEN

Monitoring

1 Million Messwerte pro Tag verarbeiten

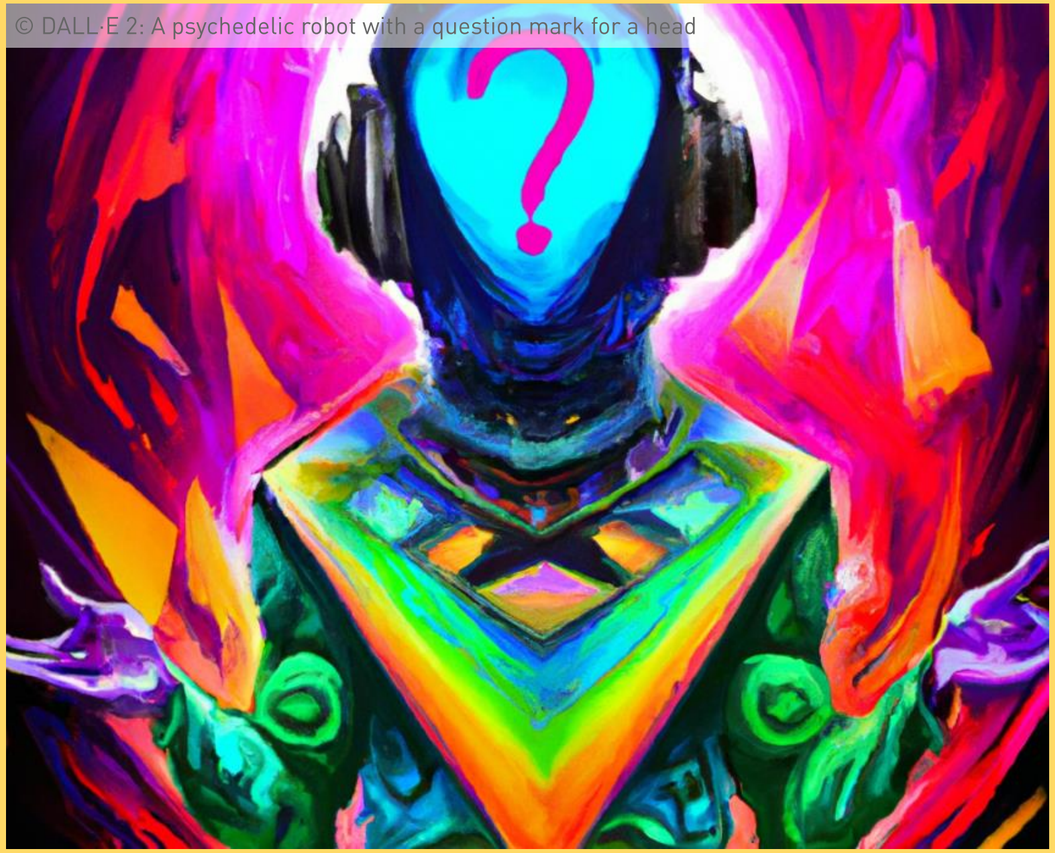
BAUELEMENTE

BIM-Modell

50.000 Elemente prüfen

Hörsaalfrage

Wie oft müssten Sie print() von Hand aufschreiben,
um 10.000 Bodenproben auszuwerten?


© DALL·E 2: A psychedelic robot with a question mark for a head

Schleifen lesen — nicht nur schreiben

Wenn KI eine Schleife generiert, stellen Sie drei Fragen:

1. Worüber?

Liste, Dict, Zahlenfolge oder Bedingung?

Erkennen

2. Was ändert sich?

Welche Variablen ändern sich pro Iteration wie?

Verfolgen (Spur-Tabelle)

3. Terminiert sie?

Gibt es eine klare Abbruchbedingung?

Prüfen

Variablen aus Schleifen sind auch außerhalb der Schleife verfügbar. Python kennt zwei Schleifen: `for` und `while`.

WIEDERHOLUNGEN MIT BEKANNTER ANZAHL



© DALL·E 2: Infinite loop of intricate swirling mandala patterns

FOR-SCHLEIFE ÜBER EINE SEQUENZ

Die for-Schleife wiederholt einen Block für alle Elemente einer Sequenz. `e` wird dabei immer mit dem aktuellen Element belegt.

```
for Element in Sequenz:
    # Statement
```

```
seq = ['a','b','c']
for e in seq:
    print(e)
```

Ausgabe:

```
a
b
c
```

FOR-SCHLEIFE MIT ZÄHLER

Klassisch n-mal wiederholen: `range(n)` erzeugt die Folge 0 bis n–1.

```
n = 3
seq = range(n)
for e in seq:
    print(e)
```

Ausgabe:

```
0
1
2
```

Dictionaries iterieren

Einfaches `for key in seq:` liefert nur die Schlüssel.

```
seq = {"key1": "wert1", "key2": "wert2"}
for key in seq:
    print(key)
```

Ausgabe:

```
key1
key2
```

Mit `items()` kommen Schlüssel und Wert gemeinsam.

```
for key, wert in seq.items():
    print(key, wert)
```

Ausgabe:

```
key1 wert1
key2 wert2
```

for-Schleife Praxisbeispiel

Umrechnung einer Liste an Messwerten von imperialen Fuß in metrische Meter.

```
measurements_feet = [4.2, 2.3, 6.2, 10.5]
measurements_meter = []

for feets in measurements_feet:
    measurements_meter.append(0.3048 * feets)

print(measurements_meter)
```

Ausgabe:

```
[1.2801600000000002, 0.70104, 1.88976, 3.2004]
```

Kurzschreibweise (List Comprehension):

```
[0.3048 * f for f in measurements_feet]
```

Spur-Tabelle: KI-Loop lesen

KI-generierter Code — *erst lesen, dann ausführen!*

```
# AI generated
temperaturen = [18.5, 21.3, 19.8, 22.1]
summe = 0
for t in temperaturen:
    summe = summe + t
durchschnitt = summe / len(temperaturen)
print(durchschnitt)
```

Worüber iteriert die Schleife? Wie verändert sich `summe` pro Iteration?

Spur-Tabelle auf Papier ausfüllen:

Iter.	`t`	`summe`
1	?	?
2	?	?
3	?	?
4	?	?
Ende	—	?

Spur-Tabelle: Lösung

```
temperaturen = [18.5, 21.3, 19.8, 22.1]
summe = 0
for t in temperaturen:
    summe = summe + t
durchschnitt = summe / len(temperaturen)
print(durchschnitt)
```

Ausgabe:

```
20.425
```

Iter.	`t`	`summe`
1	18.5	18.5
2	21.3	39.8
3	19.8	59.6
4	22.1	81.7
Ende	—	81.7

81,7 / 4 = 20,425

KI-Loop verifizieren

Drei Schritte, wenn KI eine Schleife generiert:

1. Identifizieren

Worüber iteriert die Schleife? *(Liste, Dict, Zahlenfolge?)*

2. Verfolgen

Die ersten zwei und die letzte Iteration durchlaufen.

3. Prüfen

Terminiert die Schleife? Gibt es eine klare Abbruchbedingung?

Merksatz

Nicht sofort ausführen: erst lesen, die ersten zwei und die letzte Iteration verfolgen, dann Terminierung prüfen.

WIEDERHOLUNGEN MIT UNBEKANNTER ANZAHL

while-Schleife

Die Bedingung wird vor jedem Durchlauf geprüft. Ist sie von Anfang an falsch, läuft der Block nie.

```
while Bedingung:
    # Statement
```

- `True` : Block läuft einmal, danach wird wieder geprüft
- `False` : Schleife endet, der Code läuft nicht (noch einmal)

while vs. Do-While vs. Repeat-Until

Do-While und Repeat-Until sind in Python konzeptionelle Muster, die mit `while` nachgebildet werden.

while

Bedingung kann von Anfang an falsch sein: null Durchläufe.

```
Bedingung = True/False
while Bedingung:
    # Statement
    Bedingung = False
```

Do-While

Bedingung wird am Ende geprüft: mindestens ein Durchlauf.

```
Bedingung = True
while Bedingung:
    # Statement
    Bedingung = False
```

Repeat-Until

Wiederholt, bis die Bedingung wahr wird: mindestens ein Durchlauf.

```
Bedingung = False
while not Bedingung:
    # Statement
    Bedingung = True
```

Endlosschleifen erkennen

KI generiert gelegentlich Endlosschleifen. Diese Zeichen verraten sie:

- Loop-Variable wird im Schleifenblock nicht verändert
- Bedingung kann logisch nie `False` werden
- Kein `break` und kein Zähler als Sicherheitsnetz

> ⚠ **Achtung**
>
> While-Schleifen ohne sichtbare Abbruchbedingung sind ein Risiko, besonders in KI-generiertem Code. Immer die Terminierung prüfen.

```
x = 10
while x > 0:        # Bedingung scheinbar sinnvoll
    print(x)        # ABER: x wird nie verändert
                    # → ENDLOSSCHLEIFE
```

Endlosschleifen absichern

While-Schleifen haben keine maximale Wiederholungsanzahl. Sicherheitsbedingung immer hinzufügen:

Endlosschleife: Bedingung ändert sich nie

```python
Bedingung = True
while Bedingung:
    # Statement
    Bedingung = True
```

Absicherung mit maximaler Iterationsanzahl

```python
max_iter = 1000
iterations = 0
while bedingung and iterations < max_iter:
    # Statement
    iterations += 1
```

\+ Schützt vor hängendem Programm

– Pflicht bei KI-generierten `while`-Schleifen ohne klare Terminierung

Schleife abbrechen und Elemente überspringen

Gesucht: das erste essbare Ding im Rucksack.

```
dinge_in_meinem_rucksack = ["Papier", "Stift", "Apfel", "Brot", "Messer"]
essbares = {"Apfel", "Brot"}
```

`break` beendet die Schleife sofort:

```
essbar = None
for ding in dinge_in_meinem_rucksack:
    if ding in essbares:
        essbar = ding
        break
print(essbar)   # Apfel
```

`continue` springt zur nächsten Iteration:

```
for ding in dinge_in_meinem_rucksack:
    if ding not in essbares:
        continue
    print(f"Essbar: {ding}")
```

Endlosschleife mit Timeout-Abbruch

Mit `break` lassen sich zusätzliche Abbruchbedingungen realisieren, wie z.B. eine maximale Ausführzeit:

```python
import time

iterations = 0
start = time.time()

while True:  # DAS IST EINE ENDLOSSCHLEIFE
    iterations += 1
    elapsed = time.time() - start

    if elapsed > 3:
        print(f"Timeout after {iterations} iterations")
        break
```

Programmflusskontrolle

Wir kennen nun zwei Werkzeuge:

Bedingte Ausführung

- `if`, `else`, `elif`

Mehrfachausführung

- `while`, `for`

Jedes Programm, was jemals auf einem Computer ausgeführt wurde, könnte mit diesen Mitteln geschrieben werden.

→ Schleifen als Algorithmus-Bausteine: siehe `09a_Algorithmen` (S09)

QUESTIONS